Things needed for Assignment 1

Things needed for Assignment 1

Topic	Item	Function
1. Imports	`import java.util.*;`	ArrayList, Collections, StringTokenizer
	`import java.io.*;`	BufferedReader, PrintWriter
2. `BufferedReader br = new BufferedReader(new InputStreamReader(System.in));`	`br.readLine()`	Reads full line as String
	`Integer.parseInt(s)`	String → int
	`Long.parseLong(s)`	String → long
	`Double.parseDouble(s)`	String → double
3. `PrintWriter out = new PrintWriter(System.out);`	`out.println(x)`	Print with newline
	`out.print(x)`	Print without newline
	`out.flush()`	Force output
4. `StringTokenizer st = new StringTokenizer(s, " ");`	`st.nextToken()`	Get next word/value
	`st.hasMoreTokens()`	Check remaining tokens
	`countTokens()`	How many tokens left
5. `.split(“ “);`	`String[] st=s.split(“ “);`	Separates and splits

Topic	Item	Function
6. `Collections.sort(list, comparator);`	**Use Case**	Faster than split() Negative → keep Positive → swap
	`Collections.sort(list)`	Default ascending
7. Character Utility	`Character.isLowerCase(c)`	Lowercase check
	`Character.isUpperCase(c)`	Uppercase check
	`Character.isDigit(c)`	Digit check
8. String Useful Helpers	`s.length()`	String length
	`s.charAt(i)`	Character at index
	`s.toCharArray()`	Convert to char array
	`s.equals(t)`	Safe comparison
9. `ArrayList<type> arr = new ArrayList<>();`	`arr.add(x)`	Add element
	`arr.add(i, x)`	Add at index
	`arr.get(i)`	Access
	`arr.set(i, x)`	Replace
	`arr.remove(i)`	Remove
	`arr.size()`	Current size
	`arr.clear()`	Remove all
	`arr.contains(x)`	Check existence

Topic	Item	Function
10. `ArrayList<int[]> pairs = new ArrayList<>();`	`pairs.add(new int[]{a, b})`	Store pair
11. Loops (Very Common)	`for(int i=0;i<n;i++)`	Standard loop
	`for(int x : arr)`	Enhanced loop
12. Useful Math Helpers	`Math.max(a,b)`	Maximum
	`Math.min(a,b)`	Minimum
	`Math.abs(x)`	Absolute value

Assignment

A. Odd or Even?

time limit per test: 1 second
memory limit per test: 256 megabytes

Forbidden Keywords for the Quiz: sort, open, file, creat(, fstream, thread, process, system(, exec(

Do you know how to tell if a number is **Odd** or **Even**? You are given T numbers, and for each of those numbers, you have to tell whether the number is odd or even.

Input

The first line will contain a single integer T ($1 \leq T \leq 100$). Each of the next T lines will contain a number N ($-10^5 \leq N \leq 10^5$).

Output

For each N, you have to print whether the number is odd or even. See the sample input-output format to know what exactly you have to print.

```
import java.io.*;
public class Assignment1 {
   public static void main(String[] args) throws IOException{
      BufferedReader br= new BufferedReader(new InputStreamReader(System.in));
      PrintWriter out= new PrintWriter(System.out);

      int count=Integer.parseInt(br.readLine());

      for(int i=1; i<=count; i++){
         int N= Integer.parseInt(br.readLine());

         if(N%2==0){
            out.println(N + " is an Even number.");
         }

         else{
            out.println(N + " is an Odd number.");
         }
      }

      out.flush();
   }
}
```

B. Can you solve Arithmetic Expressions?

time limit per test: 1 second
memory limit per test: 256 megabytes

Forbidden Keywords for the Quiz: eval, sort, open, file, creat(, fstream, thread, process, system(, exec(

Can you solve arithmetic expressions with your programming knowledge? Let's find it out. You will be given some arithmetic expressions, and you have to solve them.

Input

The first line will contain a number $T(1 \leq T \leq 1000)$ representing the number of test cases. Then for each test case, you will be given an arithmetic expression. Please see the sample input below. It is guaranteed that the numbers inside the arithmetic expression will be between 1 and 1000.

Output

For each test case, you have to print the result. Look at the sample output for reference.

Important Note: Your answer might contain floating point numbers, and in that case, your answer doesn't have to be exactly equal to the actual answer. For example, if your answer is 20.250000001 and the judge's solution is 20.25, your answer will still be considered correct. As long as it is really close to the correct solution, your solution will be considered correct. Formally speaking, if your solution is x, and the judge's solution is y, then as long as $|x - y| \leq 10^{-6}$, your solution will be correct. In the above example, your solution was 20.250000001 and the judge's solution was 20.25. If you take the difference of these two numbers, they are smaller than 10^{-6}. Similarly, if the judge's solution is 19.0000000000 and your solution is 19, it is still correct, as the difference is 0, which is less than 10^{-6}.

Example

input Copy

```
15
calculate 67 + 41
```

```
import java.io.*;

public class P2 {
   public static void main(String[] args) throws IOException{
      BufferedReader br=new BufferedReader(new InputStreamReader(System.in));
      PrintWriter out=new PrintWriter(System.out);

      int n=Integer.parseInt(br.readLine());

      for(int i=0; i<n; i++){
         String s=br.readLine();
         String[] st=s.split(" ");

         float a= Float.parseFloat(st[1]);
         String b= st[2];
         float c= Float.parseFloat(st[3]);

         if(b.equals("+")) out.printf("%.6f\n", a+c);
         else if(b.equals("-")) out.printf("%.6f\n", a-c);
         else if(b.equals("*")) out.printf("%.6f\n", a*c);
         else if(b.equals("/")) out.printf("%.6f\n", a/c);
      }
```

```
        out.flush();
    }
}
```

C. Fast Sum

time limit per test: 1 second
memory limit per test: 256 megabytes

Forbidden Keywords for the Quiz: sort, open, file, creat(, fstream, thread, process, system(, exec(

Your friend is trying to solve the following problem. You are given $\mathbf{T}$ test cases. For each test case, you are given an integer $\mathbf{N}$. You have to find out the summation of 1 to N. More formally, your friend has to calcuate

$$\sum_{x=1}^{x=N} x$$

Your friend wrote the following code in Python to solve it:

```
import java.io.*;

public class A3 {
    public static void main(String[] args) throws IOException {
        BufferedReader br=new BufferedReader(new InputStreamReader(System.in));
        PrintWriter out=new PrintWriter(System.out);

        int T = Integer.parseInt(br.readLine());
        for (int t = 0; t < T; t++) {
            long N = Integer.parseInt(br.readLine());            //Long- longer number
            long sum = (N*(N+1))/2;
            out.println(sum);
        }
        out.flush();
    }
}
```

D. Is Sorted?

time limit per test: 1 second
memory limit per test: 256 megabytes

Forbidden Keywords for the Quiz: sort, open, file, creat(, fstream, thread, process, system(, exec(

You are given an array of N integers. Determine whether the given array is in non-decreasing order.

An array is said to be in non-decreasing order if, for every valid index i such that $1 \leq i < N$ (1-based indexing), the condition $A[i] \leq A[i+1]$ holds true.

Example: $[1, 2, 4, 5]$, $[1, 2, 2, 4, 4, 5]$ are in the non-decreasing order because every element is less than or equal to the one after it.

Input

The first line contains a single integer T $(1 \leq T \leq 100)$ — the number of test cases. Each testcase contains two lines.

In each test case, the first line contains a single integer N $(1 \leq N \leq 10^4)$ — the number of elements in the array. The second line contains N integers separated by spaces $a_1, a_2, a_3 \ldots a_n$ $(1 \leq a_i \leq 10^6)$ — the elements of the array.

Output

For each test case, print **YES** if the array is in non-decreasing order. Otherwise, print **NO**.

Example

input Copy

```
3
4
1 2 3 3
4
1 5 2 6
1
5
```

output Copy

```
YES
NO
YES
```

```
import java.io.*;

public class A4 {
    public static void main(String[] args) throws IOException{
        BufferedReader br=new BufferedReader(new InputStreamReader(System.in));
        PrintWriter out= new PrintWriter(System.out);

        int t=Integer.parseInt(br.readLine());
        for(int i=0; i<t; i++){
            int len=Integer.parseInt(br.readLine());
            String[] s=br.readLine().split(" ");
            boolean boo=true;


            for(int j=0; j<len-1; j++){
                int curr=Integer.parseInt(s[j]);
                int next= Integer.parseInt(s[j+1]);     //Next diye len-1 korle bhalo
                if(curr>next){
                    boo=false;
```

```
                break;
            }
        }


        if(boo==true){
            out.println("YES");
        }
        else{
            out.println("NO");
        }
    }
    out.flush();
  }
}
```

E. Reverse Sorting

time limit per test: 2 seconds
memory limit per test: 256 megabytes

Forbidden Keywords for the Quiz: sort, open, file, creat(, fstream, thread, process, system(, exec(

You are given an array A of $\mathbf{N}$ integers. Your task is to sort the array in non-decreasing order using only a specific type of operation:

- In one operation, you may select any subarray of length exactly 3 and reverse it. In other words, you choose some $1 \leq i, j \leq N$ such that $j - i + 1 = 3$ and reverse `A[i...j]`.

You can apply this operation as many times as you like (or not at all). Your goal is to determine whether it is possible to sort the array using only this operation. If it is possible, you have to print the operations too. Look at the output format for a better understanding.

Input

The first line contains an integer N $(1 \leq N \leq 1000)$. The second line contains N integers A_i $(1 \leq S_i \leq 10^5)$.

Output

If it is not possible to make the array non-decreasing, print **NO**. Otherwise print **YES**. If your answer is **YES**, you have to output the number of moves you needed, let it be M. Then the next M lines will contain pairs of (i, j) representing a valid operation. Make sure your moves are valid, and those moves make the array A non-decreasing.

NOTE: If you don't understand the output format properly, look at the sample input-output and explanation.

Examples

input Copy

```
4
2 3 1 1
```

output Copy

```
YES
2
1 3
2 4
```

```
import java.io.*;
import java.util.ArrayList;

public class A5 {
    public static void main(String[] args) throws IOException{
        BufferedReader br= new BufferedReader(new InputStreamReader(System.in));
        PrintWriter out=new PrintWriter(System.out);

        //Filling up the array
        int n=Integer.parseInt(br.readLine());
        String s=br.readLine();
        StringTokenizer st=new StringTokenizer(s, " ");    //*  */

        int[] arr=new int[n];

        for(int i=0; i<n; i++){
            arr[i]= Integer.parseInt(st.nextToken());
        }
```

```
//Initializing things to print
boolean srt_possible=true;
int moves=0;
ArrayList<int[]> moves_indx=new ArrayList<>();


//Arranging subarray
for(int i=1; i<n-1; i++){
    for(int j=1;j<n-1;j++){
        if(arr[j-1]>arr[j+1]){
        int temp= arr[j+1];
        arr[j+1]=arr[j-1];
        arr[j-1]=temp;

        moves_indx.add(new int[]{j, j+2});    //* */
        moves++;
        }
    }

}


//Check if possible
for(int i=0; i<n-1; i++){
    if(arr[i]>arr[i+1]){
        srt_possible=false;
        break;
    }
}


//print
if(srt_possible==false){
    out.println("NO");
}
else{
    out.println("YES");
    out.println(moves);
    for(int i=0; i<moves; i++){
        int[] op= moves_indx.get(i);
        out.println(op[0]+ " " + op[1]);
    }
}
out.flush();
```

```
    }
}
```

F. An Ancient Sorting Algorithm

time limit per test: 1 second
memory limit per test: 256 megabytes

Forbidden Keywords for the Quiz: sort, open, file, creat(, fstream, thread, process, system(, exec(

You are given an array of N integers. You have to sort the array in non-decreasing order using a custom sorting algorithm with the following constraint:

- You may only swap adjacent elements with the same parity (i.e., both even or both odd)

Sort the array in non-decreasing order until no more such swaps are possible and print the final array.

Input

The first line contains a single integer N $(1 \leq N \leq 1000)$ — the number of elements in the array.

The second line contains N integers separated by spaces $a_1, a_2, a_3 \ldots a_n$ $(1 \leq a_i \leq 10^6)$ — the elements of the array.

Output

Print the final array after sorting the array in non-decreasing order until no more such swaps are possible.

Examples

input Copy

```
7
4 2 4 7 1 6 1
```

output Copy

```
2 4 4 1 7 6 1
```

```
import java.io.*;
import java.util.*;

public class A6 {
    public static void main(String[] args) throws IOException{
        BufferedReader br=new BufferedReader(new InputStreamReader(System.in));
        PrintWriter out=new PrintWriter(System.out);

        int t= Integer.parseInt(br.readLine());

        //Fill array
        int[] arr=new int[t];
        String s= br.readLine();
        StringTokenizer st=new StringTokenizer(s, " ");
        for(int i=0; i<t; i++){
            arr[i]= Integer.parseInt(st.nextToken());
```

```
        }


        //Swaping
        for(int i=0;i<t; i++){
            for(int j=0; j<t-1;j++){
                if((arr[j]%2==0 && arr[j+1]%2==0) ||(arr[j]%2!=0 && arr[j+1]%2!=0)){
                    if(arr[j]> arr[j+1]){
                    int temp= arr[j];
                    arr[j]=arr[j+1];
                    arr[j+1]=temp;
                }
            }
            }
        }

        //print
        for(int i=0; i<t; i++){
            out.print(arr[i]+ " ");
        }
        out.flush();
    }

}
```

G. Sorting Again??

time limit per test: 1 second
memory limit per test: 256 megabytes

Forbidden Keywords for the Quiz: sort, open, file, creat(, fstream, thread, process, system(, exec(

Suppose you are given a task to rank the students. You have gotten the marks and ID of the students. Now your task is to rank the students based on their marks using a sorting algorithm. If two or more students get the same mark, then students with the lower ID will get prioritized. See the input and output for a better understanding.

However, **you have to keep in mind that your sorting algorithms perform the minimum number of swapping operations.**

Example

input Copy

```
2
7
7 4 9 3 2 5 1
40 50 50 20 10 10 10
3
1 2 3
40 59 10
```

output Copy

```
Minimum swaps: 4
ID: 4 Mark: 50
ID: 9 Mark: 50
ID: 7 Mark: 40
ID: 3 Mark: 20
ID: 1 Mark: 10
ID: 2 Mark: 10
ID: 5 Mark: 10
Minimum swaps: 1
ID: 2 Mark: 59
ID: 1 Mark: 40
ID: 3 Mark: 10
```

```
import java.util.*;
import java.io.*;


public class A7 {
    public static void main(String[] args) throws IOException{
        BufferedReader br=new BufferedReader(new InputStreamReader(System.in));
        PrintWriter out=new PrintWriter(System.out);


        int t=Integer.parseInt(br.readLine());

        for(int i=0; i<t; i++){
            //Filling
            int n= Integer.parseInt(br.readLine());
            String s_id= br.readLine();
            StringTokenizer st_id= new StringTokenizer(s_id, " ");
            int[] arr_id=new int[n];
```

```
        String s_marks= br.readLine();
        StringTokenizer st_marks= new StringTokenizer(s_marks, " ");
        int[] arr_marks=new int[n];


        for(int j=0; j<n; j++){
            arr_id[j]=Integer.parseInt(st_id.nextToken());
            arr_marks[j]=Integer.parseInt(st_marks.nextToken());
        }

        //Arranging
        int min_swp=selection_sort(arr_id,arr_marks);    //Selection sort= min swap


        //print
        out.println("Minimum swaps: " + min_swp);
        for(int k=0; k<n; k++){
            out.println("ID: "+ arr_id[k]+ " Mark: "+ arr_marks[k]);
        }
    }

    out.flush();
}




//Selection sort
    public static int selection_sort(int[] arr_id, int[] arr_marks){
        int min_swp=0;


        for(int i=0; i<arr_id.length; i++){
            int max_indx= i;
            for(int j=i+1; j<arr_id.length; j++){
                if((arr_marks[max_indx]< arr_marks[j])||
                (arr_marks[max_indx]== arr_marks[j] && arr_id[max_indx]> arr_id[j])){
                    max_indx=j;
                }
            }
```

```
        if(max_indx!=i){
            int temp_id= arr_id[i];
            arr_id[i]=arr_id[max_indx];
            arr_id[max_indx]= temp_id;


            int temp_marks= arr_marks[i];
            arr_marks[i]=arr_marks[max_indx];
            arr_marks[max_indx]= temp_marks;


            min_swp++;
        }
    }
    return min_swp;
  }
}
```

H. Trains 2

time limit per test: 1 second
memory limit per test: 256 megabytes

Forbidden Keywords for the Quiz: open, file, creat(, fstream, thread, process, system(, exec(

Note: You can use the built-in sort function for this problem only for this problem in this lab.

You have recently been recruited as the Software Engineer at Jumanji Railway Software System. You have a big task at hand. You will be given the departure schedules for N trains. The next N lines will contain the name of the train, the destination, and the departure time.

Sort the given lines in the **special lexicographical order** based on the name of the trains. If two or more trains have the same name, then the train with the latest departure time will get prioritized. If there is still a tie, then the train which comes first in the input will come first.

Example

input Copy

```
6
HogwartsExpress will depart for Hogwarts at 11:15
AppleExpress will depart for AppleLand at 15:00
HogwartsExpress will depart for Ilvermony at 11:10
dhumketuexpress will depart for Rajshahi at 06:00
dhumketu will depart for Rajshahi at 06:00
AppleExpress will depart for OrangeLand at 15:00
```

output Copy

```
dhumketu will depart for Rajshahi at 06:00
dhumketuexpress will depart for Rajshahi at 06:00
AppleExpress will depart for AppleLand at 15:00
AppleExpress will depart for OrangeLand at 15:00
HogwartsExpress will depart for Hogwarts at 11:15
HogwartsExpress will depart for Ilvermony at 11:10
```

Note

The **special lexicographical order** is defined as using the following alphabetical ordering:

`abcdefghijklmnopqrstuvwxyzABCDEFGHIJKLMNOPQRSTUVWXYZ`

We sort them using rules:

- Sort by train name (special order)
- Same name → latest time first
- Still same → input order

```
import java.util.*;
import java.io.*;


public class A8 {
    //Train class
    public static class Train{
        String name;
        int time;
        int indx;
```

```
    String line;

    Train(String name, int time, int indx, String line){
        this.name=name;
        this.time=time;
        this.indx=indx;
        this.line=line;
    }
}


//main method
public static void main(String[] args) throws IOException{
    BufferedReader br=new BufferedReader(new InputStreamReader(System.in));
    PrintWriter out=new PrintWriter(System.out);

    int c=Integer.parseInt(br.readLine());
    ArrayList<Train> train_list= new ArrayList<>();


    //Fill arraylist
    for(int i=0; i<c; i++){
        String line= br.readLine();
        StringTokenizer st=new StringTokenizer(line, " ");

        //First token == Name
        String name=st.nextToken();

        //Last token==time
        String time_string="";
        while(st.hasMoreTokens()){
            time_string=st.nextToken();
        }
        int time=convt_time(time_string);

        train_list.add(new Train(name, time, i, line));
    }



    //Collection.sort(-1 -> No change && 1 -> sort)
    Collections.sort(train_list, new Comparator<Train>(){
        public int compare(Train t1, Train t2){
            //name
```

```
            int comp_name=comp_Name(t1.name,t2.name);
            if(comp_name<0) return -1;
            if(comp_name>0) return 1;          //Do not use else

            //time(age boro)
            if(t1.time>t2.time) return -1;
            if(t1.time<t2.time) return 1;          //Careful with 1 and -1

            //index (Age choto)
            if(t1.indx<t2.indx) return -1;
            if(t1.indx>t2.indx) return 1;

            return 0;
        }
    });




    //print
    for(int i=0; i<train_list.size(); i++){
        out.println(train_list.get(i).line);
    }
    out.flush();
}

//time_string -> time_int
public static int convt_time(String time_string){
    String[] p=time_string.split(":");
    int hour=Integer.parseInt(p[0]);
    int min=Integer.parseInt(p[1]);

    return 60*hour + min;
}




//helper compare name
public static int comp_Name(String t1, String t2){
    int n=Math.min(t1.length(), t2.length());
    for(int i=0; i<n; i++){
        char c1=t1.charAt(i);
        char c2=t2.charAt(i);
```

```
            //Case 1: Same letter
            if(c1==c2) continue;

            //Case 2: Different letter
            boolean low1=Character.isLowerCase(c1);
            boolean low2=Character.isLowerCase(c2);


            if(low1 && low2){
                if(c1<c2) return -1;
                else return 1;
            }

            if(!low1 && !low2){
                if(c1<c2) return -1;
                else return 1;
            }

            if(low1) return -1;
            if(low2) return 1;
        }

        //Case 3: One of them is longer
        if(t1.length()<t2.length()) return -1;
        if(t1.length()>t2.length()) return 1;

        return 0;
    }





}
```